

Lista de Exercícios de Revisão — Prova 01

Sistemas Operacionais — Prof. Me. Filipo Mór

Instruções: Escolha a única alternativa correta para cada questão.

Parte 1 — Questões

1. No contexto da evolução histórica, qual foi o principal marco técnico introduzido pelo sistema CTSS (1961) que alterou a percepção de uso dos computadores da época?

- A) A substituição total de cartões perfurados por fitas magnéticas.
- B) A introdução do conceito de time-sharing, permitindo que múltiplos usuários interagissem simultaneamente com a máquina.**
- C) O desenvolvimento da primeira interface gráfica baseada em janelas e mouse.
- D) A criação do sistema de arquivos hierárquico em árvore.
- E) A obrigatoriedade do chip TPM 2.0 para garantir a segurança dos dados.

2. Sobre a diferença conceitual entre um "programa" e um "processo", é correto afirmar que:

- A) O programa é uma entidade dinâmica em execução, enquanto o processo é o código estático no disco.
- B) Ambos são sinônimos e possuem exatamente as mesmas estruturas de controle no kernel.
- C) O processo é uma instância ativa de um programa, caracterizado por um estado corrente e recursos associados.**
- D) Um programa pode ter apenas um processo associado a ele durante toda a vida útil do sistema.
- E) O processo reside exclusivamente na memória secundária, enquanto o programa reside na CPU.

3. O sistema operacional é frequentemente definido como um "Gerenciador de Recursos". Qual das alternativas abaixo descreve uma tarefa típica dessa função?

- A) Compilar o código-fonte de aplicações Java para linguagem de máquina.
- B) Decidir como e quando a CPU será alocada para diferentes processos que disputam o processador.**
- C) Desenvolver algoritmos de inteligência artificial para o usuário final.
- D) Criar layouts de design para apresentações de slides acadêmicos.
- E) Substituir fisicamente módulos de memória RAM defeituosos sem desligar o hardware.

4. Na transição da 1ª para a 2ª geração de sistemas operacionais, o surgimento do processamento em lote (batch) visava principalmente:

- A) Permitir que usuários jogassem video games em tempo real.
- B) Reduzir o tempo de setup e a intervenção humana entre a execução de diferentes tarefas (jobs).**
- C) Introduzir o uso de internet banda larga nos mainframes IBM 704.
- D) Eliminar a necessidade de um processador central (CPU).
- E) Garantir que cada usuário tivesse um terminal dedicado em sua residência.

5. O Bloco de Controle de Processo (BCP ou PCB) é a estrutura de dados que representa o processo no S.O. Qual conjunto de informações é essencialmente armazenado no BCP para permitir a interrupção e posterior retomada de um processo?

- A) Apenas o nome do executável e a data de criação do arquivo.
B) O histórico de navegação na web e as senhas do usuário.
C) O contexto de execução, incluindo registradores como Program Counter (PC) e Stack Pointer (SP).
D) O código-fonte completo em linguagem de alto nível (ex: C ou Python).
E) A lista de todos os dispositivos de hardware conectados via USB.
6. No modelo de 5 estados de um processo, qual evento causa a transição do estado "Running" (Em Execução) para o estado "Blocked" (Bloqueado)?
A) A expiração do time-slice (fatia de tempo) definida pelo escalonador.
B) Uma solicitação de operação de Entrada/Saída (E/S), como a leitura de um arquivo em disco.
C) A decisão do escalonador de curto prazo em favorecer um processo mais prioritário que acabou de chegar.
D) O término da execução do programa (chamada exit).
E) A criação de um processo filho através da chamada fork().
- Nota: As alternativas A e C descrevem a transição Running → Ready (preempção por quantum esgotado ou por processo mais prioritário). A alternativa D descreve a transição Running → Exit/Terminated. Esses três casos são conceitualmente distintos de Running → Blocked e representam boas situações para estudo do diagrama de estados completo.*
7. Considere a chamada de sistema fork() no padrão Unix. Se um programador executa o comando pid = fork();, o que acontece com a variável pid no processo filho recém-criado, caso a operação seja bem-sucedida?
A) Ela recebe o valor do Identificador de Processo (PID) do pai.
B) Ela recebe um valor negativo, indicando erro.
C) Ela recebe o valor 0 (zero).
D) Ela recebe o mesmo valor PID do próprio processo filho.
E) Ela permanece nula até que o processo pai execute um wait().
- Nota: Atenção à distinção: o processo FILHO recebe pid = 0; o processo PAI recebe pid = PID do filho (número positivo); em caso de erro, o pai recebe pid = -1. A alternativa D descreve o que o PAI recebe, não o filho. Esse é um ponto de confusão frequente em provas.*
8. O que caracteriza um processo do tipo "CPU-Bound"?
A) Ele realiza frequentes chamadas ao sistema para leitura e escrita em dispositivos periféricos.
B) Ele passa a maior parte do seu tempo no estado "Blocked" aguardando interrupções de hardware.
C) Ele utiliza intensivamente o processador para cálculos e processamento de dados, realizando poucas operações de E/S.
D) Ele é automaticamente finalizado pelo S.O. por consumir muita energia.
E) Ele nunca entra na fila de prontos (Ready Queue), indo direto para a execução.
9. Sobre a chamada de sistema exec(), qual é o seu efeito primordial em um processo Unix?
A) Criar uma cópia idêntica do processo atual.
B) Suspende o processo pai até que o filho termine.
C) Substituir a imagem do processo corrente (código, dados, pilha) por um novo programa executável.
D) Alterar a prioridade de escalonamento para o nível de tempo real.
E) Enviar um sinal de terminação para o processo init.

10. O conceito de "Overhead" na troca de contexto refere-se a:

- A) O tempo que a CPU gasta executando instruções úteis dos programas de usuário.
- B) O ganho de velocidade obtido ao aumentar o número de núcleos do processador.
- C) O tempo gasto pelo sistema operacional em tarefas de gerência (como salvar e restaurar registradores) que não contribui diretamente para o progresso do job do usuário.**
- D) A capacidade máxima de armazenamento de um disco rígido SSD.
- E) A prioridade máxima que um processo pode atingir através da técnica de aging.

11. No algoritmo de escalonamento Shortest Job First (SJF) na versão preemptiva (também conhecido como SRTF - Shortest Remaining Time First), o que ocorre quando um novo processo chega à fila de prontos com um tempo de execução estimado menor que o tempo restante do processo que está atualmente na CPU?

- A) O processo atual continua até terminar, e o novo processo espera na fila.
- B) O novo processo é ignorado, pois o SJF não permite preempção.
- C) Ocorre a preempção do processo atual, e o novo processo assume a CPU imediatamente.**
- D) O escalonador divide o tempo da CPU igualmente entre os dois processos (Round-Robin).
- E) O processo atual é movido para o estado "Exit" permanentemente.

Nota: A preempção no SRTF ocorre somente se o tempo restante estimado do novo processo for estritamente menor que o tempo restante do processo atual. Em caso de empate, o comportamento depende da implementação — em geral, o processo em execução continua. Esse detalhe é importante na resolução de diagramas de Gantt.

12. Qual é a principal vantagem da utilização do algoritmo Round-Robin em sistemas de tempo compartilhado?

- A) Garantir que processos CPU-bound terminem o mais rápido possível, sem interrupções.
- B) Fornecer tempos de resposta aceitáveis para todos os usuários, evitando que um único processo monopolize a CPU.**
- C) Minimizar o número de trocas de contexto no sistema para reduzir o overhead.
- D) Eliminar completamente a necessidade de uma fila de processos bloqueados.
- E) Priorizar processos com base no nome do usuário que os iniciou.

13. O problema do "Starvation" (Inanição) em algoritmos de prioridade fixa ocorre quando:

- A) A CPU superaquece devido ao excesso de cálculos matemáticos.
- B) Processos de baixa prioridade nunca conseguem executar porque sempre existem processos de maior prioridade na fila de prontos.**
- C) O disco rígido fica cheio e não permite mais o swap-out de processos.
- D) Um processo entra em loop infinito e trava todo o sistema operacional.
- E) O escalonador de curto prazo é mais lento que o escalonador de longo prazo.

14. A técnica de "Aging" (Envelhecimento) é utilizada para solucionar o problema de starvation. Ela consiste em:

- A) Reduzir gradualmente o tempo de CPU (quantum) de processos antigos.
- B) Mover processos muito velhos para a memória secundária permanentemente.
- C) Aumentar gradualmente a prioridade de um processo à medida que ele permanece esperando na fila de prontos por muito tempo.**
- D) Diminuir a prioridade de processos que utilizam muita memória RAM.

E) Reiniciar o computador automaticamente a cada 24 horas de uso contínuo.

15. No escalonamento Multinível com Feedback (MLFQ), por que um processo geralmente é movido para uma fila de menor prioridade após consumir toda a sua fatia de tempo (quantum)?

A) Para punir o usuário por rodar programas pesados.

B) Para identificar processos que são I/O-bound e dar-lhes mais CPU.

C) Para penalizar processos CPU-bound (longos), favorecendo processos curtos ou interativos que tendem a liberar a CPU antes do fim do quantum.

D) Porque filas de menor prioridade possuem fatias de tempo menores, acelerando a execução. *[INCORRETA]*

E) Para liberar espaço no cache L1 do processador.

Nota: A alternativa D está errada: no MLFQ clássico, filas de MENOR prioridade possuem quanta MAIORES (ex: fila 1 = 8ms, fila 2 = 16ms, fila 3 = 32ms). Isso ocorre porque processos CPU-bound, que "caem" para filas inferiores, precisam de mais tempo contínuo de CPU por execução, embora sejam escalonados com menor frequência.

16. O fenômeno da "Fragmentação Interna" no particionamento fixo de memória ocorre quando:

A) O espaço livre entre duas partições ocupadas é pequeno demais para qualquer novo processo.

B) O S.O. não consegue encontrar nenhuma partição disponível para um novo job.

C) Um processo é carregado em uma partição maior do que o necessário, resultando em espaço desperdiçado dentro daquela partição.

D) O processo tenta acessar um endereço de memória que pertence ao núcleo (kernel).

E) A tabela de páginas do processo cresce além do limite físico da memória RAM.

17. No contexto de particionamento dinâmico, qual é o objetivo da técnica de "Compactação"?

A) Reduzir o tamanho dos arquivos executáveis no disco para economizar espaço.

B) Deslocar os processos na memória principal para agrupar todos os pequenos buracos livres em um único bloco contíguo de memória.

C) Comprimir os dados do usuário usando algoritmos como ZIP ou RAR enquanto estão na RAM.

D) Dividir um processo em páginas de tamanho fixo para facilitar a alocação.

E) Aumentar a velocidade do barramento de dados entre a CPU e o disco.

18. Considere o algoritmo de alocação "Best-Fit". Como ele decide em qual bloco de memória livre alocar um novo processo?

A) Ele escolhe o primeiro bloco encontrado que seja grande o suficiente.

B) Ele escolhe o bloco cujo tamanho seja o mais próximo possível do tamanho solicitado, minimizando o sobra de espaço.

C) Ele sempre escolhe o maior bloco disponível para permitir que o processo cresça no futuro.

D) Ele escolhe o bloco que foi liberado há menos tempo pelo sistema operacional.

E) Ele divide a memória ao meio e aloca o processo na primeira metade disponível.

19. O conceito de "Memória Virtual" permite que um sistema execute programas maiores que a memória física disponível. Isso é possível principalmente através da:

A) Utilização exclusiva de registradores da CPU para armazenar variáveis globais.

B) Paginação por demanda, onde apenas as partes do programa necessárias no momento são carregadas na RAM, mantendo o restante no disco.

C) Eliminação da pilha (stack) e do heap do espaço de endereçamento do processo.

- D) Desativação do modo supervisor do processador durante a execução de aplicações pesadas.
- E) Replicação física de pentes de memória RAM através de software.

Nota: A paginação por demanda é o mecanismo mais comum, mas memória virtual pode ser implementada também por segmentação (como no Intel 80286) ou por segmentação combinada com paginação (80386+). A alternativa B está correta no contexto da prova; para maior rigor, o enunciado poderia incluir "principalmente".

20. O estado de "Thrashing" em sistemas de paginação é caracterizado por:

- A) Uma taxa altíssima de utilização da CPU devido ao processamento paralelo.
- B) O sistema gastar mais tempo trocando páginas entre a RAM e o disco do que executando instruções úteis, levando a uma queda drástica na performance.**
- C) O desligamento súbito do hardware por excesso de temperatura nos módulos de memória.
- D) A corrupção total do sistema de arquivos após uma falha de energia.
- E) O processo conseguir acessar endereços de outros processos sem permissão.

21. No modelo de estados de processo, o que diferencia fundamentalmente o estado "Pronto" (Ready) do estado "Bloqueado" (Blocked)?

- A) O processo no estado "Pronto" está na memória secundária, enquanto o "Bloqueado" está na CPU.
- B) O processo "Pronto" possui todos os recursos necessários para executar, exceto a CPU, enquanto o "Bloqueado" aguarda um evento externo ou recurso adicional.**
- C) O estado "Pronto" é exclusivo de processos do sistema (kernel), e o "Bloqueado" é exclusivo de processos do usuário.
- D) A transição de "Pronto" para "Execução" é feita pelo usuário, enquanto a de "Bloqueado" para "Execução" é automática.
- E) Um processo "Pronto" nunca consome espaço na Tabela de Processos do sistema operacional.

22. Considere a execução de múltiplas threads em um ambiente de exclusão mútua. O que define uma "Condição de Corrida" (Race Condition)?

- A) Quando dois processos terminam exatamente ao mesmo tempo, causando um erro no escalonador.
- B) Uma situação onde o resultado final da execução depende da ordem específica em que o acesso a dados compartilhados é intercalado, na ausência de sincronização adequada.**
- C) O momento em que a CPU atinge 100% de uso e começa a superaquecer.
- D) Quando o processo pai termina antes do processo filho, transformando o filho em um processo "zumbi".
- E) A disputa entre o teclado e o mouse pela interrupção de hardware de maior prioridade.

Nota: A definição completa exige mencionar a ausência de sincronização: com mecanismos de sincronização adequados (semáforos, mutexes), pode haver intercalamento de threads sem race condition. A alternativa B foi corrigida para incluir esse elemento.

23. Sobre o fenômeno do Deadlock (Impasse), qual das alternativas descreve a condição de "Espera Circular"?

- A) Um processo aguarda por si mesmo em um loop infinito de código.

B) O escalonador Round-Robin distribui o tempo de CPU de forma circular entre todos os processos.

C) Existe uma cadeia de processos onde cada processo detém um recurso solicitado pelo próximo processo na cadeia, e o último aguarda um recurso detido pelo primeiro.

D) O sistema operacional reinicia o contador de programa (PC) sempre que detecta um erro de segmentação.

E) É a técnica de mover processos da memória para o disco e vice-versa para evitar fragmentação.

24. No contexto de sincronização, o que é uma "Região Crítica"?

A) Uma área do disco rígido onde o setor de boot do sistema operacional está localizado.

B) O intervalo de tempo em que a CPU está realizando uma troca de contexto (overhead).

C) Um segmento de código onde um processo acessa recursos compartilhados que não devem ser acessados simultaneamente por outros processos.

D) A fila de processos de alta prioridade em um escalonador de tempo real.

E) Uma instrução de hardware que só pode ser executada em modo usuário.

25. Qual estratégia de tratamento de Deadlock é aplicada pelo algoritmo do Banqueiro (Banker's Algorithm)?

A) Detecção e Recuperação: permite o deadlock ocorrer e depois o desfaz matando processos.

B) Avestruz: ignora o problema, assumindo que ele ocorre raramente.

C) Prevenção: elimina uma das quatro condições de Coffman (como a posse e espera).

D) Evitação (Avoidance): o sistema analisa cada solicitação de recurso e só a concede se o sistema permanecer em um "estado seguro".

E) Preempção de Recursos: retira à força um recurso de um processo para entregá-lo a outro de maior prioridade.

***Nota:** O Algoritmo do Banqueiro exige que cada processo declare previamente sua demanda MÁXIMA de recursos (vetor Max). Essa exigência é uma limitação prática importante, pois em sistemas reais os processos raramente conhecem sua necessidade máxima com antecedência.*

Parte 2 — Gabarito Comentado

1. Resposta: B

O CTSS foi pioneiro ao permitir que múltiplos usuários compartilhassem o tempo da CPU por meio de sessões interativas, rompendo com o modelo exclusivo do processamento em lote para um único usuário por vez.

2. Resposta: C

O programa é uma entidade estática — instruções armazenadas em disco. O processo é a abstração dinâmica da execução: possui estado (ex: Running, Blocked), contexto de execução e recursos associados. Um mesmo programa pode originar múltiplos processos simultâneos.

3. Resposta: B

Uma das funções primordiais do S.O. é a gerência de recursos. O escalonador de CPU é o mecanismo responsável por alocar o recurso "processador" entre os processos que o disputam.

4. Resposta: B

Sistemas de lote automatizavam a carga de jobs em sequência, evitando que a CPU ficasse ociosa enquanto o operador humano configurava manualmente a máquina. O objetivo central era minimizar o tempo ocioso do processador.

5. Resposta: C

Para que a troca de contexto seja possível, o S.O. precisa salvar no BCP o Program Counter (onde o programa parou), o Stack Pointer e os demais registradores da CPU, permitindo que a execução seja retomada exatamente de onde foi interrompida.

6. Resposta: B

A transição Running → Blocked ocorre quando o processo solicita uma operação de E/S e não pode prosseguir até que o dado esteja disponível. Atenção às demais transições: a expiração do quantum e a chegada de processo mais prioritário levam a Running → Ready; o término do programa leva a Running → Exit.

7. Resposta: C

No Unix, fork() possui retornos distintos conforme o contexto: o PROCESSO FILHO recebe pid = 0; o PROCESSO PAI recebe pid = PID do filho (valor positivo); em caso de erro (falha na criação), o pai recebe pid = -1 e nenhum filho é criado. A alternativa D descreve o que o pai recebe, não o filho.

8. Resposta: C

Processos CPU-Bound têm longos surtos (bursts) de processamento e raramente solicitam E/S, passando a maior parte do tempo no estado Running ou Ready. Em contrapartida, processos I/O-Bound realizam frequentes chamadas de E/S e passam a maior parte do tempo no estado Blocked.

9. Resposta: C

Enquanto fork() cria uma cópia do processo atual, exec() substitui completamente a imagem do processo corrente (código, dados, pilha e heap) por um novo programa executável, mantendo o mesmo PID.

10. Resposta: C

Overhead é o custo operacional da gerência. Durante a troca de contexto, a CPU não executa instruções úteis para o usuário — ela salva e restaura registradores, atualiza o BCP e realiza operações internas do S.O. Minimizar o overhead de troca de contexto é um objetivo de projeto dos sistemas operacionais.

11. Resposta: C

O SRTF é a versão preemptiva do SJF e garante o menor tempo médio de espera entre os algoritmos de escalonamento. A preempção ocorre somente quando o tempo restante estimado do processo recém-chegado é estritamente menor que o tempo restante do processo em execução. Em caso de empate, o processo em execução geralmente continua.

12. Resposta: B

O Round-Robin distribui a CPU em fatias de tempo iguais (quantums) entre todos os processos. Isso impede que processos longos monopolizem a CPU e garante tempos de resposta aceitáveis para sistemas interativos, criando a ilusão de execução simultânea.

13. Resposta: B

Starvation (inanição) ocorre em escalonamento por prioridade fixa quando o fluxo de processos de alta prioridade é constante, impedindo indefinidamente que processos de baixa prioridade recebam tempo de CPU.

14. Resposta: C

O Aging resolve o starvation aumentando gradualmente a prioridade de processos que aguardam há muito tempo na fila de prontos, garantindo que, mesmo processos de baixíssima prioridade, eventualmente consigam executar.

15. Resposta: C

Se um processo consome todo o seu quantum, é provável que seja CPU-bound. Ao rebaixá-lo para uma fila de menor prioridade, o MLFQ favorece processos curtos e interativos, que tendem a liberar a CPU voluntariamente antes do fim do quantum. Importante: no MLFQ clássico, filas de menor prioridade possuem quantums MAIORES (não menores), pois processos CPU-bound precisam de mais tempo contínuo por execução.

16. Resposta: C

No particionamento fixo, se uma partição tem 100 KB e o processo ocupa 80 KB, os 20 KB restantes dentro daquela partição ficam desperdiçados — isso é Fragmentação Interna. Já a Fragmentação Externa (alternativa A) é característica do particionamento dinâmico, onde os buracos livres entre partições são pequenos demais para novos processos.

17. Resposta: B

A compactação resolve a Fragmentação Externa no particionamento dinâmico, deslocando todos os processos para uma extremidade da memória e consolidando os buracos em um único bloco contíguo livre. É uma operação cara em termos de tempo e exige relocação dinâmica.

18. Resposta: B

O Best-Fit percorre toda a lista de blocos livres e seleciona o menor bloco que ainda seja suficiente para o processo, minimizando o desperdício interno. Como desvantagem, tende a criar muitos fragmentos residuais pequenos e exige varredura completa da lista.

19. Resposta: B

A paginação por demanda é o mecanismo mais comum de implementação de memória virtual: somente as páginas realmente necessárias em cada momento são carregadas na RAM, e o restante permanece no disco. O acesso a uma página ausente gera um page fault, que aciona o carregamento pelo S.O. Outros mecanismos de memória virtual incluem segmentação e segmentação combinada com paginação.

20. Resposta: B

Thrashing ocorre quando o conjunto de trabalho (working set) dos processos ativos supera a memória física disponível, fazendo com que o sistema passe a maior parte do tempo substituindo páginas entre RAM e disco em vez de executar instruções. O resultado é uma queda drástica na utilização efetiva da CPU.

21. Resposta: B

Ambos os estados representam processos "parados", mas por razões distintas: o processo Ready possui todos os recursos necessários e está esperando apenas que o escalonador lhe conceda a CPU. O processo Blocked não pode executar mesmo que a CPU esteja livre, pois aguarda um evento externo (término de E/S, sinal, liberação de recurso).

22. Resposta: B

A condição de corrida ocorre quando o resultado final de uma computação depende da ordem de intercalação das operações entre threads concorrentes sobre dados compartilhados, na ausência de sincronização adequada. Com mecanismos de sincronização corretos (semáforos, mutexes), o intercalamento pode ocorrer sem gerar inconsistências.

23. Resposta: C

A espera circular é uma das quatro condições de Coffman necessárias para o deadlock. Ela forma um ciclo fechado de dependências: P1 aguarda recurso de P2, P2 aguarda de P3, ..., Pn aguarda recurso de P1. Sem intervenção externa (preempção ou cancelamento de processo), nenhum progresso é possível.

24. Resposta: C

A Região Crítica (ou Seção Crítica) é o trecho de código que manipula dados compartilhados entre processos ou threads concorrentes. Para evitar inconsistências causadas por condições de corrida, apenas um processo deve executá-la por vez, utilizando primitivas como semáforos, mutexes ou monitores.

25. Resposta: D

O Algoritmo do Banqueiro implementa a estratégia de Evitação de Deadlock: antes de conceder um recurso, o sistema simula a alocação e verifica se o estado resultante é seguro (se existe uma sequência de execução que permita todos os processos terminarem). Se o estado for inseguro, a solicitação é negada. Limitação prática importante: o algoritmo exige que cada processo declare previamente sua demanda máxima de recursos (vetor Max), o que raramente é possível em sistemas reais.

— Fim do documento —